

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files

print("Libraries imported successfully")
```

Libraries imported successfully

```
uploaded = files.upload()

video_filename = list(uploaded.keys())[0]

print("Video uploaded successfully:")
print(video_filename)
```

Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving 15205993_2160_3840_30fps.mp4 to 15205993_2160_3840_30fps.mp4

Video uploaded successfully:
15205993_2160_3840_30fps.mp4

```
video = cv2.VideoCapture(video_filename)

if not video.isOpened():
    print("Error: Could not open video.")
else:
    ret1, frame1 = video.read()
    ret2, frame2 = video.read()

    video.release()

    if not ret1 or not ret2:
        print("Error: Could not read two consecutive frames.")
    else:
        print("Two consecutive frames read successfully.")

        gray1 = cv2.cvtColor(
            frame1,
            cv2.COLOR_BGR2GRAY
        )

        gray2 = cv2.cvtColor(
            frame2,
            cv2.COLOR_BGR2GRAY
        )

        print("Frames converted to grayscale ")
```

Two consecutive frames read successfully.
Frames converted to grayscale.

```
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(
    cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB)
)
plt.title("First Frame")
plt.axis("off")

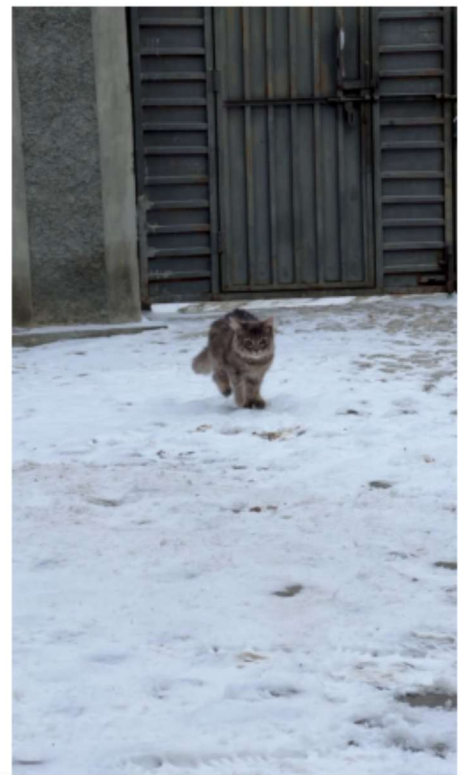
plt.subplot(1, 2, 2)
plt.imshow(
    cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB)
)
plt.title("Second Frame")
plt.axis("off")

plt.tight_layout()
plt.show()
```

First Frame



Second Frame



```
# Detect feature points in the first frame
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
```

```

        blockSize=7
    )
    points_old = cv2.goodFeaturesToTrack(
        gray1,
        mask=None,
        **feature_params
    )
    if points_old is None:
        print("No feature points detected.")
    else:
        print("Feature points detected:", len(points_old))

    # Lucas-Kanade parameters

    lk_params = dict(
        winSize=(15, 15),
        maxLevel=2,
        criteria=(
            cv2.TERM_CRITERIA_EPS |
            cv2.TERM_CRITERIA_COUNT,
            10,
            0.03
        )
    )
    points_new, status, error = cv2.calcOpticalFlowPyrLK(
        gray1,
        gray2,
        points_old,
        None,
        **lk_params
    )

    good_new = points_new[status == 1]
    good_old = points_old[status == 1]

    print("Successfully tracked points:", len(good_new))

```

```

Feature points detected: 100
Successfully tracked points: 100

```

```

lucas_kanade = frame2.copy()

for new, old in zip(good_new, good_old):

    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()

    x_new = int(x_new)
    y_new = int(y_new)

    x_old = int(x_old)

```

```
y_old = int(y_old)

# Draw motion arrow

cv2.arrowedLine(
    lucas_kanade,
    (x_old, y_old),
    (x_new, y_new),
    (255, 0, 0),
    2,
    tipLength=0.3
)

# Draw feature point

cv2.circle(
    lucas_kanade,
    (x_new, y_new),
    4,
    (0, 255, 0),
    -1
)

plt.figure(figsize=(8, 6))

plt.imshow(
    cv2.cvtColor(
        lucas_kanade,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")
plt.show()
```

Lucas-Kanade Sparse Optical Flow



```
flow = cv2.calcOpticalFlowFarneback(  
    gray1,  
    gray2,  
    None,  
    0.5,  
    3,  
    15,  
    3,  
    5,  
    1.2,  
    0  
)  
  
print("Farneback Dense Optical Flow calculated.")
```

Farneback Dense Optical Flow calculated.